



GoalOS
Signoff Pro

INSTITUTIONAL RESEARCH

PUBLIC-ALPHA BOUNDARY

BLOCKCHAIN CREDIBILITY

Publication control

Elite Institutional Edition v2.0
July 1, 2026

Core standard

No Proof.
No Trust.
No Settlement.

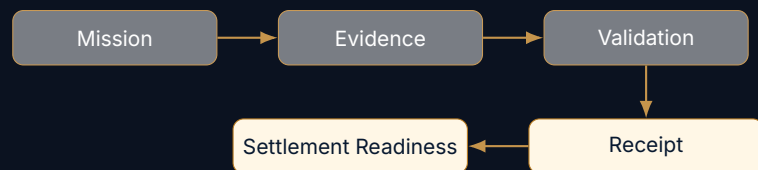
Proof Before Settlement

The GoalOS Signoff Pro Standard for Blockchain Credibility

Evidence Dockets, Replayable Receipts, Human Authority, and the Proof Package Mandate

Blockchain proves the transaction. GoalOS proves the work.

A serious blockchain project should show the proof package before asking for trust, payment, governance authority, or public credibility.



Research artifact prepared for institutional communication, technical review, blockchain governance, DAO due diligence, AI-agent settlement design, and public trust standards.

Executive tear sheet

The paper in one page

Executive position

Strategic claim. Blockchains are excellent at proving that transactions happened. They do not automatically prove that the work behind those transactions was real, complete, reviewed, safe, or worthy of settlement. GoalOS Signoff Pro supplies the missing credibility layer: mission definition, evidence mapping, replay, validation, risk disclosure, human authority, signed receipt, and optional public commitment.

Institutional message

Blockchain proves the transaction. GoalOS proves the work.

Standard

No Proof. No Trust. No Settlement.

Primary audience

DAOs, foundations, treasury councils, grant programs, auditors, exchanges, enterprises, AI-agent operators, investors, and protocol governors.

Core object

A public-safe proof package: Mission Contract, Evidence Docket, ProofBundle, Replay Log, Validator Report, Risk Ledger, Human Authority Record, Mission Receipt.

What v28 adds

The *Blockchain Credibility Standard Lab*: a flagship public demo for proof-before-settlement.

What v29 adds

The *Blockchain Proof Mandate and Due Diligence Lab*: a practical demand checklist that asks every project, "Where is the proof package?"

Boundary

Public demos are proof-architecture demonstrations: no wallets, no payments, no custody, no user data, no Mainnet settlement claim.

Mandate language

Recommended public demand. Before a blockchain project asks for treasury funds, governance approval, milestone payment, audit closure, AI-agent compensation, RWA credibility, or ecosystem trust, require a GoalOS-style proof package. Not hype. Not dashboard screenshots. Not a vague update. The proof package.

Publication boundary

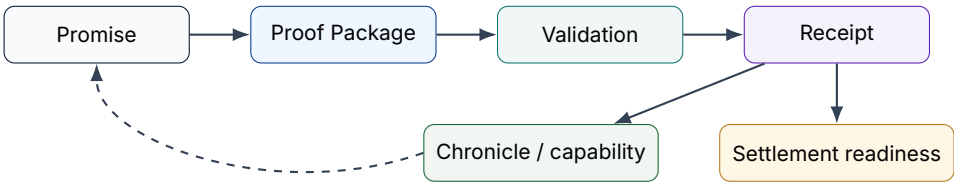
Claim-bounded, public-safe, and adoption-oriented

Claim boundary

This paper is a research and institutional communication artifact. It is not legal, financial, tax, investment, compliance, cybersecurity certification, external audit, or production-deployment advice. It does not claim live Mainnet settlement, custody, token sale, wallet activation, payment routing, achieved AGI, achieved ASI, or external factual certification. It specifies a proof architecture and public standard for deciding when off-chain work is mature enough to influence settlement, governance, reputation, or future capability.

Evidence note

Grounding. The live GoalOS Signoff Pro site presents a proof-to-acceptance console with six gates and public browser-local demos [1]. Mission 001 exposes a reproducibility packet containing mission contract, environment, task fixtures, B0–B6 baselines, runner config, proof bundle, replay log, ledgers, verifier report, scoreboard, claims matrix, and replay path [2]. The public data posture states no forms, input fields, upload path, wallet connection, payments, cookies, or analytics [3]. The proof-to-settlement lab explicitly describes simulation mode: no wallet, no escrow, no payment, no value moved [4].



Only accepted proof should become settlement signal, public credibility, governance authority, or reusable institutional memory.

Abstract

Blockchains provide programmable settlement, public state, and tamper-resistant transaction history. They do not, by themselves, prove that the off-chain work behind a transaction was real, complete, independently reviewed, risk-bounded, or worthy of payment. This paper formalizes that missing layer as the *work-verification gap*. We propose GoalOS Signoff Pro as a proof-to-acceptance and proof-to-settlement architecture for blockchain credibility: a system that converts work claims into mission contracts, evidence dockets, proof bundles, replay logs, validator reports, risk ledgers, human-authority decisions, signed receipts, challenge windows, and optional public anchors. The paper provides a formal object model, life-cycle state machine, acceptance predicate, settlement-safety invariant, threat model, claim-maturity lattice, due-diligence scorecard, and adoption roadmap. It also specifies two flagship public demonstrations: v28, the Blockchain Credibility Standard Lab, and v29, the Blockchain Proof Mandate and Due Diligence Lab. The result is a concise institutional standard: *Blockchain proves the transaction; GoalOS proves the work*. Therefore, for serious blockchain projects: *No Proof. No Trust. No Settlement*.

Core thesis

Contribution. GoalOS Signoff Pro is not another blockchain, wallet, oracle, audit badge, or dashboard. It is the proof-and-signoff layer that tells low-trust ecosystems when a claim has matured enough to justify settlement, reputation, governance, or future capability.

Contents

1	Executive summary	1
2	Problem: the work-verification gap	2
2.1	Settlement is not acceptance	2
2.2	Why informal proof fails at scale	2
2.3	Blockchain projects are especially exposed	3
3	Design thesis	3
3.1	Non-goals	4
4	Related work	4
4.1	Blockchains and programmable settlement	4
4.2	Oracles and hybrid smart contracts	4
4.3	Provenance, auditability, and reproducibility	4
4.4	DAOs and governance	5
5	The GoalOS proof object model	5
6	Formal architecture	6
6.1	Participants	6
6.2	State machine	6
6.3	Acceptance predicate	7

6.4	Claim maturity lattice	7
6.5	Proof debt	7
6.6	Economic control interface	8
7	Protocol: Proof-to-Settlement	8
7.1	Plain-English protocol	8
7.2	Protocol sketch	8
7.3	On-chain and off-chain split	9
8	Blockchain applications	9
8.1	DAO grants	9
8.2	Audit remediation	10
8.3	Protocol upgrades	10
8.4	AI-agent settlement	10
8.5	Real-world asset and oracle-adjacent workflows	10
9	v28 and v29: public demonstration architecture	10
9.1	v28: Blockchain Credibility Standard Lab	10
9.2	v29: Blockchain Proof Mandate and Due Diligence Lab	11
9.3	v28/v29 as a communication stack	11
10	Threat model	11
10.1	Adversaries	11
10.2	Safety properties	12
11	Implementation blueprint	13
11.1	Minimal viable public standard	13
11.2	GitHub-native implementation	13
11.3	Maturity levels	13
11.4	Production hardening	14
12	Communication and adoption strategy	14
12.1	Public language	14
12.2	Stakeholder-specific ask	15
12.3	Inevitable framing	15
13	Evaluation plan	15
13.1	Research questions	15
13.2	Metrics	16

14 Limitations	16
15 Conclusion	16
A Institutional adoption blueprint	18
A.1 The board-level decision	18
A.2 The credibility ladder	18
A.3 The public expectation	19
B Reference proof package schema	19
C Due-diligence scorecard	20
D Template mandate clauses	20
D.1 DAO grant clause	20
D.2 Treasury payment clause	20
D.3 Protocol upgrade clause	20
E Public copy bank	21
F Production-readiness checklist	21

1 Executive summary

Operational standard

Blockchain proves the transaction. GoalOS proves the work.

No Proof. No Trust. No Settlement.

Blockchain projects succeed or fail on credibility. The protocol may be transparent, the contract may be deployed, and the transaction may be public, but the work behind the transaction is often still opaque. Was the grant milestone actually completed? Was the audit issue really remediated? Is the upgrade ready? Did the AI agent perform the assigned task? Is the RWA claim supported? Did the vendor deliver what the treasury is about to pay for?

The core answer of this paper is that blockchain credibility needs a proof institution. A proof institution turns off-chain work into structured, inspectable, replayable, signed objects. It is not enough to say that value moved. The ecosystem needs to know why value deserved to move.

GoalOS Signoff Pro provides that institutional grammar. Its public product language already centers on acceptance: define done, bind evidence, preserve judgment, and issue receipts that executives, reviewers, and systems can independently inspect [1]. This paper extends that grammar into a blockchain standard.

The standard is intentionally simple for public adoption:

Require the proof package before trust, payment, governance, or reputation moves.

Why this is strategically important

Blockchains made settlement programmable. They did not make off-chain work self-verifying. That gap leaves room for milestone ambiguity, grant abuse, audit theater, governance confusion, unauditable AI-agent outputs, and irreversible payments based on insufficient evidence. A GoalOS-style proof package gives every serious stakeholder a practical demand: show the mission, evidence, replay path, validation, risk ledger, human decision, and receipt.

What this paper delivers

- A formal definition of the work-verification gap.
- A GoalOS proof object model for blockchain contexts.
- A proof-gated lifecycle from mission to settlement readiness.
- A settlement-safety invariant: value should not move unless acceptance evidence is mature.
- Public demonstration specs for v28 and v29.
- Practical adoption templates for DAOs, treasuries, grants, audits, AI-agent settlement, protocol upgrades, and investor due diligence.

Executive position

Bottom line. Most blockchain projects that ask for public trust should be expected to publish a GoalOS-style proof package. Not because it guarantees success, but because it materially improves credibility, auditability, accountability,

and settlement discipline.

2 Problem: the work-verification gap

2.1 Settlement is not acceptance

Let a blockchain transaction be represented as a state transition

$$\delta : (S_t, x, \sigma) \rightarrow S_{t+1},$$

where S_t is current chain state, x is transaction data, and σ is authorization material such as signatures. The chain can verify that δ is syntactically valid under protocol rules. It can verify signatures, balances, nonces, contract preconditions, gas constraints, and deterministic computation.

But many economically important blockchain decisions depend on a semantic predicate over external work:

$$W(M, O, E, R, V, H) \in \{0, 1\},$$

where M is a mission or obligation, O is output, E is evidence, R is replay or reproducibility material, V is validator review, and H is human authority. The chain cannot directly observe W unless a trustworthy representation of W is supplied. This is a broader cousin of the oracle problem. Ethereum's developer documentation describes the oracle problem as the challenge that oracle data must be correct for smart contracts to execute correctly, and that trusting oracle operators can undermine the trustless aspect of smart contracts [5]. The work-verification gap extends that issue from external data to external labor, deliverables, audits, governance claims, and AI-agent outputs.

Definition 2.1 (Work-verification gap). *The work-verification gap is the distance between (i) an on-chain rule that can settle a value-bearing state transition and (ii) the off-chain institutional evidence required to know whether the real-world or agentic work motivating that transition has been completed, validated, and accepted.*

Failure mode

The blind spot. A DAO can vote to pay a grant; an escrow can release; a token can move; a protocol upgrade can execute. None of these events alone proves that the underlying work was complete, safe, replayable, or accepted by the right authority.

2.2 Why informal proof fails at scale

Informal proof works until projects become high-stakes, multi-party, cross-jurisdictional, or adversarial. At scale, the following failure modes recur:

1. **Milestone ambiguity.** The project cannot determine whether the promised deliverable was achieved because “done” was never formalized.
2. **Evidence fragmentation.** Relevant evidence is scattered across GitHub, chats, dashboards, PDFs, tweets, audits, and private drives.
3. **Output/authority confusion.** A persuasive report is mistaken for accepted work.
4. **Replay failure.** Reviewers cannot reproduce or inspect the claimed result.

5. **Reviewer opacity.** It is unclear who validated what, under which criteria, and with what conflicts.
6. **Risk laundering.** Known caveats disappear as the claim moves from technical review to public communication.
7. **Settlement irreversibility.** Funds, reputation, governance weight, or public claims move before evidence is mature.
8. **Memory contamination.** Unsupported work influences future missions, model routing, or protocol decisions.

2.3 Blockchain projects are especially exposed

Blockchain projects often combine low-trust coordination with irreversible consequences. Contributors may not know each other. DAO delegates may review complex claims under time pressure. Grant programs need to fund work without creating rent-seeking. Protocol upgrades can affect users who did not participate in the decision. Oracles and bridges depend on external assumptions. AI agents may produce outputs that look plausible but are difficult to verify. In this environment, trust should not depend on charisma, marketing, dashboard aesthetics, or social momentum. It should depend on a proof package.

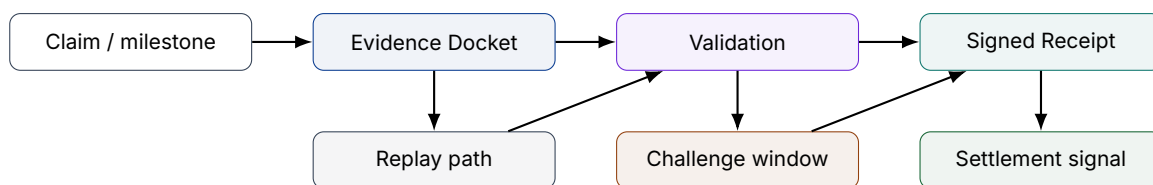
3 Design thesis

Public positioning

Public thesis. Most serious blockchain projects should be expected to publish a GoalOS-style proof package before asking for public trust, treasury funds, governance approval, ecosystem credibility, or settlement. The simplest demand is: *Where is the proof package?*

The design thesis is that blockchain credibility requires three layers:

1. **Computational settlement.** The chain executes deterministic rules and preserves public state.
2. **Proof institution.** GoalOS structures the off-chain work into inspectable proof packages.
3. **Authority and memory.** Accepted proof becomes a receipt, an optional anchor, and a controlled source of future capability or reputation.



GoalOS does not make every claim true. It makes the path from claim to acceptance explicit, inspectable, replayable where possible, and safe to reference.

Figure 1: GoalOS as the missing institutional layer between claims and settlement.

3.1 Non-goals

The architecture is deliberately bounded. It is not a claim that public demos already perform live settlement. It is not a wallet, broker, exchange, custodian, token-sale mechanism, legal certification engine, or source of investment advice. The live Signoff Pro site states a no-data public posture, and its \$AGIALPHA boundary page says the token is external and not sold, issued, brokered, custodied, distributed, redeemed, staked, or made available by GoalOS, MontrealAI, or QuebecAI [3, 6]. The AGIJobManager Ascension public site similarly states that public pages are read-only and do not connect wallets, approve tokens, broadcast transactions, authorize funds, certify external factual correctness, or grant production authority [7].

Claim boundary

Research boundary. This paper specifies a proof architecture and demonstration standard. It does not claim live economic activation. The strongest current claim is: GoalOS demonstrates a public-alpha proof-to-acceptance architecture and can be extended into a proof-to-settlement standard when external review, challenge windows, anchoring, and production authorization are independently established.

4 Related work

4.1 Blockchains and programmable settlement

Bitcoin introduced a peer-to-peer electronic cash system using proof-of-work consensus to order transactions without a central intermediary [8]. Ethereum generalized blockchain execution through smart contracts and decentralized applications [9]. These systems made settlement programmable, but they did not eliminate the need to reason about external facts and work.

4.2 Oracles and hybrid smart contracts

Oracles address the fact that smart contracts require access to external data and systems. Ethereum documentation frames the oracle problem as the need for oracle data to be correct, available, and incentive-compatible [5]. Chainlink 2.0 expands the oracle idea toward decentralized oracle networks that provide external connectivity and off-chain computation for hybrid smart contracts [10]. GoalOS is complementary: it does not merely provide data to a contract; it structures the evidence, validation, replay, risk, and human acceptance surrounding work.

4.3 Provenance, auditability, and reproducibility

W3C PROV supplies a data model for representing entities, activities, agents, and their provenance relationships [11]. Modern software supply-chain frameworks such as SLSA, in-toto, and SPDX provide lessons for artifact integrity, attestation, and dependency transparency [12, 13, 14]. NIST's AI Risk Management Framework emphasizes governance, mapping, measuring, and managing AI risk [15]. NIST's Secure Software Development Framework captures practices for reducing software vulnerabilities and improving traceability [16]. GoalOS imports this general discipline into blockchain and agentic work: not just code provenance, but mission provenance; not just dependency transparency, but claim transparency; not just audit logs, but acceptance receipts.

4.4 DAOs and governance

DAOs attempt to coordinate decisions and resources through tokenized or cryptographic governance. Their weakness is often not the absence of a vote, but the absence of a high-quality evidentiary object before a vote. A GoalOS proof package can act as the evidence object that makes DAO votes, grants, and treasury disbursements more credible.

5 The GoalOS proof object model

This section defines the core artifacts. The terminology is designed to be precise enough for engineers and obvious enough for non-technical stakeholders.

Definition 5.1 (Mission Contract). *A Mission Contract is a bounded specification of work. It defines objective, scope, success criteria, failure criteria, constraints, risk class, allowed actions, denied actions, evidence requirements, reviewer roles, budget limits, challenge window, and done condition.*

Definition 5.2 (Evidence Docket). *An Evidence Docket is a structured proof room that maps claims to evidence, replay material, provenance, validator notes, risk status, acceptance decisions, and public/private boundaries.*

Definition 5.3 (ProofBundle). *A ProofBundle is a compact cryptographic and procedural summary of a work execution: trace roots, output hashes, input commitments, environment commitments, policy decisions, cost and latency records, signatures, and any replay material required by the mission.*

Definition 5.4 (Mission Receipt). *A Mission Receipt is an immutable acceptance record. It references the mission, evidence docket, validation state, human decision, timestamp, signature metadata, and optional on-chain anchor. It is the settlement-adjacent object, not the raw output.*

Definition 5.5 (Chronicle). *A Chronicle is controlled institutional memory. Only accepted proof may become memory, reusable capability, reputation, or future routing authority. Rejected, challenged, or unsupported outputs may become warnings but not authority.*

Table 1: GoalOS proof object taxonomy.

Object	Question answered	Typical content	Settlement role
Mission Contract	What exactly was promised?	Objective, criteria, scope, risk, budget, allowed actions	Defines settlement conditions
Evidence Docket	What proof exists?	Claims, artifacts, mappings, provenance, caveats	Enables review
ProofBundle	What can be cryptographically and procedurally checked?	Hashes, trace roots, signatures, environment commitments	Enables replay/anchoring
Replay Log	Can the result be reproduced or inspected?	Deterministic stages, commands, validators, outputs	Blocks unsupported claims
Validator Report	Who reviewed what?	Verdicts, disagreements, quorum, challenge outcomes	Provides confidence and accountability

Object	Question answered	Typical content	Settlement role
Risk Ledger	What could still go wrong?	Safety events, policy issues, residual risk, rollback status	Controls release class
Human Authority Record	Who accepted it?	Signoff identity, role, times-tamp, decision reason	Preserves authority boundary
Mission Receipt	What exact object can systems reference?	Acceptance hash, evidence hash, decision state, signature	Settlement-adjacent receipt
Anchor	What public commitment exists?	Receipt hash, CID, timestamp, revocation state	Tamper-evident reference
Chronicle Entry	What may influence future work?	Accepted lessons, warnings, capability updates	Controls compound-ing

6 Formal architecture

6.1 Participants

Let the system include the following actors:

$$\mathcal{A} = \{q, w, v, h, t, c\},$$

where q is a requester, w is a worker or agent, v is a validator set, h is a human authority, t is a treasury or settlement system, and c is a chain or public commitment layer.

Actors are not assumed to trust each other. The system aims to make trust decisions explicit through artifacts, signatures, quorum rules, challenge windows, and public commitments.

6.2 State machine

A mission is represented by a state variable over proof-gated lifecycle states:

$$s \in \{\text{draft, commissioned, executed, docketed, review, challenged, accepted, receipted, settlement_ready, chronicled, rejected}\}.$$

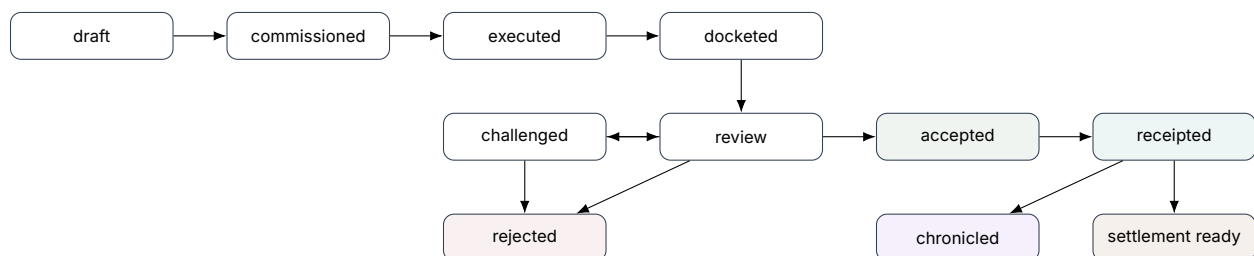


Figure 2: Mission lifecycle as a proof-gated state machine.

6.3 Acceptance predicate

Let M be a mission contract, D an evidence docket, P a proof bundle, L a replay log, R a risk ledger, V a validator report, H a human-authority record, and Γ a policy context. We define the acceptance predicate:

$$\text{Accept}(M, D, P, L, R, V, H; \Gamma) = 1$$

iff all required gates are satisfied:

$$\begin{aligned} \text{Complete}(D, M) &= 1, \\ \text{MappedClaims}(D, M) &= 1, \\ \text{Replayable}(P, L, M) &= 1, \\ \text{ValidatorQuorum}(V, \Gamma) &= 1, \\ \text{RiskWithinBoundary}(R, M, \Gamma) &= 1, \\ \text{ChallengeCleared}(D, V, \Gamma) &= 1, \\ \text{HumanAuthority}(H, M, \Gamma) &= 1. \end{aligned}$$

If any gate fails, the mission may remain useful as a draft, warning, or partial artifact, but it does not become settlement-ready.

Theorem 6.1 (Settlement safety invariant). *Assume the acceptance predicate above is a necessary precondition for every settlement transition. Then no reachable settlement-ready state exists for a mission that lacks complete evidence, mapped claims, replay material required by policy, validator quorum, risk clearance, challenge clearance, or required human authority.*

Proof. By construction, the transition relation into `settlement_ready` contains `Accept` as a necessary precondition. `Accept` is a conjunction of the seven gate predicates above. If any gate fails, the conjunction evaluates to 0, so the transition is disabled. Therefore no execution trace can reach `settlement_ready` unless all required gates hold. This is a protocol safety result, not an empirical claim that the evidence is externally true. $\square$

6.4 Claim maturity lattice

Claims should not be binary. They mature through proof states. The path can be read in two lines:

$$\begin{aligned} &\text{unsupported} \prec \text{docketed} \prec \text{mapped} \prec \text{replayed} \prec \text{validated} \\ &\prec \text{challenge_cleared} \prec \text{accepted} \prec \text{receipted} \prec \text{settlement_ready} \prec \text{reusable}. \end{aligned}$$

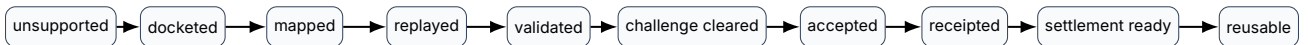


Figure 3: A claim-maturity lattice prevents premature settlement and premature memory.

6.5 Proof debt

We define proof debt as the residual deficit between required proof and available proof. Let $G = \{g_1, \dots, g_n\}$ be required gates with weights ω_i , and let $z_i \in \{0, 1\}$ indicate whether gate g_i is satisfied. Proof debt is

$$\text{Debt}(M, D, P) = \sum_{i=1}^n \omega_i (1 - z_i).$$

Settlement readiness requires $\text{Debt} = 0$ for mandatory gates and $\text{Debt} \leq \tau$ for optional or risk-adjusted gates. The public interface can express this in plain language: what is still missing before anyone should trust the claim?

6.6 Economic control interface

Let $Q(P)$ be proof quality, $U(M)$ mission utility, $K(M)$ risk class, $C(P)$ cost, and $D(P)$ proof debt. A synthetic proof-work-unit score can be represented as

$$\alpha\text{WU}(M, P) = \lambda_1 U(M)Q(P) - \lambda_2 D(P) - \lambda_3 K(M) - \lambda_4 C(P),$$

where $\lambda_i \geq 0$ are policy weights. This score should not be confused with live token settlement. It is an internal control signal that can inform budgets, validator intensity, challenge windows, and readiness decisions.

7 Protocol: Proof-to-Settlement

7.1 Plain-English protocol

1. **Request.** A requester states the objective and risk class.
2. **Mission.** GoalOS converts the objective into a Mission Contract.
3. **Work.** Workers or agents execute within bounded allowed actions.
4. **Proof.** Outputs are packaged with evidence, traces, hashes, costs, and risks.
5. **Docket.** Claims are mapped to evidence and caveats.
6. **Replay.** Reviewers attempt reproduction or inspect replay commitments.
7. **Validate.** Validators render verdicts under a quorum rule.
8. **Challenge.** The system opens a challenge window for contested claims.
9. **Authorize.** Human authority accepts, rejects, or holds the mission.
10. **Receipt.** A signed receipt is issued for accepted work.
11. **Anchor.** Optional hash anchoring creates tamper-evident public reference.
12. **Settle.** Treasury, escrow, governance, reputation, or routing may act on the receipt.
13. **Chronicle.** Accepted proof becomes controlled memory or reusable capability.

7.2 Protocol sketch

Listing 1: GoalOS proof-to-settlement protocol sketch.

```

1 def proof_to_settlement(request, policy):
2     mission = compile_mission_contract(request, policy)
3     work = execute_bounded_work(mission)
4     proof = build_proof_bundle(work, mission)
5     docket = map_claims_to_evidence(mission, work, proof)
6     replay = run_or_record_replay(docket, mission)
7     risk = produce_risk_ledger(docket, replay, policy)
8     validation = collect_validator_report(docket, replay, risk, policy)
9     challenge = run_challenge_window(docket, validation, policy)
10    authority = collect_human_authority(mission, docket, validation, risk)
11
12    if accept(mission, docket, proof, replay, risk, validation, challenge, authority):
13        receipt = issue_signed_receipt(mission, docket, validation, authority)
14        anchor = maybe_anchor_receipt_hash(receipt, policy)

```

```

15     return settlement_ready(receipt, anchor)
16
17     return reject_or_hold(mission, docket, validation, risk, challenge)

```

7.3 On-chain and off-chain split

A credible implementation should put rich evidence off-chain and compact commitments on-chain.

Table 2: Recommended off-chain/on-chain split.

Layer	Store off-chain	Commit on-chain
Mission	Full contract, criteria, risk class	Mission hash, version, policy root
Evidence	Files, reviews, logs, redactions	Docket hash, content identifier, times-tamp
Replay	Full replay logs and deterministic artifacts	Replay root, environment root
Validation	Validator notes, conflicts, evidence mapping	Validator report hash, quorum flag
Authority	Human decision rationale	Receipt hash, signer identity commitment
Settlement	Treasury context, payment authorization docs	Settlement-ready receipt hash, revocation status
Chronicle	Lessons, reusable capability package	Chronicle entry hash, capability version hash

Proposition 7.1 (Anchoring is not truth). *A receipt hash anchored on-chain makes a particular receipt tamper-evident and time-referenced. It does not by itself prove that the underlying work is true, complete, safe, or legally sufficient.*

Proof. A hash commitment binds to bytes, not to external semantic truth. Truth claims require evidence, replay, validation, challenge, risk review, and authority. Anchoring strengthens integrity of the accepted record; it does not replace validation. □

8 Blockchain applications

8.1 DAO grants

DAO grant programs are the cleanest early use case. A grant proposal can be converted into a Mission Contract. Each milestone produces an Evidence Docket. Reviewers validate outputs. The DAO or grant committee accepts, holds, or rejects the milestone. A Mission Receipt becomes the object referenced by treasury action.

Operational standard

DAO grant rule. No milestone payout without a proof package: mission, evidence, replay or inspection route, validator review, risk ledger, human or committee signoff, and receipt.

8.2 Audit remediation

Audit remediation often fails because teams claim issues are fixed but reviewers lack a structured packet connecting findings to patches, tests, replay logs, and residual risk. GoalOS can map each finding to a remediation mission, proof bundle, test evidence, reviewer verdict, and final authority.

8.3 Protocol upgrades

Protocol upgrades should be gated by evidence, simulation, rollback analysis, challenge windows, and human/governance authority. GoalOS can provide a release-readiness docket that distinguishes a persuasive upgrade narrative from an accepted upgrade proof.

8.4 AI-agent settlement

Agentic systems increase the urgency of proof-gated settlement. If agents can generate work, request funds, trigger workflows, or influence governance, then output-only acceptance is unsafe. The rule becomes: no ProofBundle, no agent settlement signal; no replay, no propagation; no rollback, no release.

8.5 Real-world asset and oracle-adjacent workflows

For RWA and oracle-adjacent systems, GoalOS can structure the evidence around provenance, attestations, redactions, challenge windows, and claim boundaries. It should not claim to solve every external truth problem. It should make the truth claim accountable and inspectable before settlement depends on it.

9 v28 and v29: public demonstration architecture

9.1 v28: Blockchain Credibility Standard Lab

The v28 lab should be the public flagship for blockchain audiences. Its purpose is to make the standard unforgettable:

Blockchain proves the transaction. GoalOS proves the work.
Do not just move value. Prove it was earned.

Required v28 sections

1. **The trust gap.** Show why transaction transparency is not work verification.
2. **The proof package.** Show mission, evidence, replay, validation, risk, human signoff, receipt, and anchor.
3. **Use-case switcher.** DAO grant, audit remediation, protocol upgrade, AI-agent work, RWA claim, partnership milestone.
4. **Settlement readiness console.** Show why output-only claims fail and proof-cleared claims pass.
5. **Boundary panel.** State no wallet, no payment, no Mainnet transaction, no legal/financial advice, zero value moved.
6. **Downloadable artifacts.** Provide public-safe JSON bundles and a public proof-dashboard schema.

9.2 v29: Blockchain Proof Mandate and Due Diligence Lab

The v29 lab should convert v28's thesis into a user action: require the proof package.

Where is the proof package?
No proof package, no serious trust.

Required v29 sections

1. **Stakeholder selector.** User, DAO delegate, grant committee, auditor, investor, partner, enterprise buyer, exchange, AI-agent operator.
2. **Proof request card.** A one-screen checklist for asking a blockchain project to prove its claim.
3. **Due diligence scorecard.** Score evidence completeness, replayability, validator independence, risk disclosure, authority, and receipt integrity.
4. **Mandate templates.** DAO policy, grant RFP clause, treasury payment rule, exchange review clause, partner procurement clause.
5. **Decision tree.** Accept, hold, challenge, reject, or request missing evidence.
6. **Public copy bank.** Social, README, docs, and governance wording.

9.3 v28/v29 as a communication stack

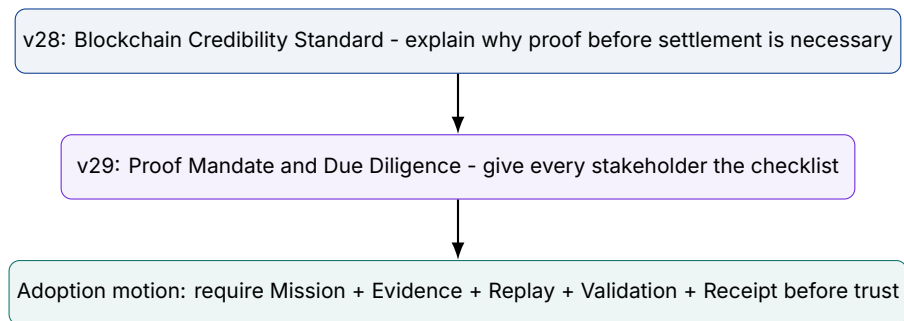


Figure 4: v28 makes the idea obvious; v29 makes the demand operational.

10 Threat model

10.1 Adversaries

The system should assume adversarial or misaligned behavior from workers, requesters, validators, governance voters, data providers, and external systems.

Table 3: Threat model and GoalOS controls.

Threat	Attack	Control
Output laundering	Persuasive narrative without proof	Mandatory docket and claim-evidence mapping
Evidence stuffing	Large irrelevant artifact dump	Criteria mapping, reviewer burden score, proof debt
Replay gap	Hashes exist but no reproduction route	Replay gate and independent replay status
Validator capture	Colluding or low-quality reviewers	Quorum rules, conflict disclosure, challenge windows
Risk hiding	Known caveats omitted from public claim	Risk ledger and claim-boundary firewall
Premature settlement	Funds move before review completes	Settlement safety invariant
Privacy leakage	Sensitive execution details exposed	Redaction maps and public/private proof boundary
Governance theater	Vote proceeds without evidence	Proof package mandate and due-diligence scorecard
Memory contamination	Failed claims influence future work	Chronicle rule: accepted proof only
Token confusion	Demo mistaken for live financial rail	Boundary pages, no-wallet demos, synthetic signals only

10.2 Safety properties

A production-grade proof-to-settlement system should target the following properties:

- **Integrity.** Receipts bind to exact evidence and decision state.
- **Replayability.** Required claims expose reproduction or inspection routes.
- **Accountability.** Reviewers, validators, and authorities are recorded.
- **Minimal disclosure.** Public proof commitments do not leak private execution details.
- **Revocability.** Receipts can be superseded or revoked when evidence fails.
- **Challengeability.** Claims can be contested before settlement or propagation.
- **Human boundary.** High-risk work preserves explicit human authority.
- **No silent compounding.** Only accepted proof influences future work.

11 Implementation blueprint

11.1 Minimal viable public standard

A project can start without live wallets or settlement. The public standard requires only:

1. Mission Contract template.
2. Evidence Docket template.
3. Claims Matrix.
4. Replay or inspection checklist.
5. Validator report.
6. Risk ledger.
7. Human signoff record.
8. Signed Mission Receipt.
9. Optional hash anchor.
10. Public proof dashboard.

11.2 GitHub-native implementation

A practical route for public projects is GitHub-native:

- Store public-safe mission and proof artifacts in version control.
- Generate proof pages with GitHub Actions.
- Publish static proof dashboards to GitHub Pages.
- Produce machine-readable JSON receipts.
- Gate settlement-related PRs on validation checks.
- Use branch protection for proof artifacts.

The current GoalOS Signoff Pro repository is already positioned as a public demo layer for acceptance, evidence review, signed receipts, optional verified receipts, browser-local demos, sample artifacts, and GitHub Actions generation [17, 18].

11.3 Maturity levels

Table 4: GoalOS blockchain credibility maturity model.

Level	Name	Capability	Public claim
0	Narrative	Project describes work with no evidence package	Marketing only
1	Docketed	Claims mapped to public-safe evidence	Reviewable claim

Level	Name	Capability	Public claim
2	Replayable	Replay or inspection path exists	Reproducible or inspectable
3	Validated	Independent validator report attached	Validator-reviewed
4	Receipted	Human authority signs acceptance receipt	Institutionally accepted
5	Anchored	Receipt hash or CID publicly committed	Tamper-evident record
6	Settlement-ready	Policy allows treasury/governance action	Ready for settlement decision
7	Chronicled	Accepted proof updates future capability	Reusable institutional memory

11.4 Production hardening

Before live settlement, the following gates should be mandatory:

1. Independent security review of receipt signing and anchoring.
2. Separation of demo code from production authorization code.
3. Key management, rotation, and revocation policy.
4. Validator identity and conflict-of-interest framework.
5. Challenge-window policy and appeal path.
6. Privacy review for evidence redaction.
7. Legal review for each jurisdiction and asset class.
8. Incident response and rollback procedure.
9. Rate limits, anti-spam, Sybil resistance, and abuse prevention.
10. Clear public labels distinguishing demo, simulation, testnet, and mainnet.

12 Communication and adoption strategy

12.1 Public language

The standard must be memorable. The following phrases should be used consistently:

- Blockchain proves the transaction. GoalOS proves the work.
- No Proof. No Trust. No Settlement.
- Do not just move value. Prove it was earned.
- Where is the proof package?
- No Evidence Docket, no strong public claim.

- No ProofBundle, no settlement signal.
- No receipt, no credibility.

12.2 Stakeholder-specific ask

Table 5: Stakeholder proof mandate.

Stakeholder	Question to ask	Expected artifact
User	Why should I trust this claim?	Public proof package
DAO delegate	What evidence supports this vote?	Governance evidence docket
Grant committee	Was the milestone earned?	Milestone receipt
Treasury council	What justifies payment?	Acceptance receipt and risk ledger
Auditor	Was remediation verified?	Finding-to-fix proof matrix
Investor	What is real versus announced?	Due-diligence scorecard
Exchange or platform	What public record exists?	Proof dashboard and receipt anchor
Enterprise buyer	Can procurement inspect delivery?	Criteria-mapped acceptance package
AI-agent operator	Did the agent earn settlement?	ProofBundle and replay log

12.3 Inevitable framing

The strongest argument is not that GoalOS is fashionable. It is that blockchain maturity requires a missing layer. Public blockchains made transactions transparent. The next credibility frontier is work transparency: evidence, replay, validation, authority, and receipts. As more value moves through DAOs, AI agents, protocol upgrades, bridges, RWAs, and ecosystem grants, informal proof will become inadequate.

Public positioning

Adoption sentence. GoalOS should become a basic expectation for serious blockchain projects: if a project asks for funds, governance approval, public legitimacy, or settlement, it should provide a proof package first.

13 Evaluation plan

Because the current public posture is demonstration-first and claim-bounded, evaluation should focus on reproducibility, user comprehension, and decision quality before live settlement.

13.1 Research questions

1. Does a GoalOS proof package reduce ambiguity in grant milestone review?
2. Do reviewers make more consistent accept/hold/reject decisions when claims are evidence-mapped?
3. Does the proof-debt metric predict review friction?
4. Does independent replay catch defects missed by output-only review?

5. Do stakeholders understand the difference between a demo, a testnet simulation, and live settlement?
6. Does public anchoring improve tamper detection without increasing privacy leakage?
7. Does Chronicle discipline prevent unsupported outputs from influencing future work?

13.2 Metrics

Table 6: Evaluation metrics.

Metric	Definition	Interpretation
Evidence completeness	Required evidence fields satisfied	Readiness of docket
Replay success rate	Percent of replayable claims reproduced	Reproducibility confidence
Validator agreement	Inter-validator consistency	Review reliability
Challenge yield	Defects discovered during challenge window	Adversarial value
Proof debt burn-down	Missing gates over time	Mission progress
Decision latency	Time from docket to decision	Process efficiency
Settlement reversals	Accepted receipts later revoked	Quality risk
Privacy incidents	Sensitive data exposed in public proof	Boundary failure
User comprehension	Stakeholders correctly explain status	Communication quality

14 Limitations

GoalOS cannot eliminate the need for judgment. It cannot make false evidence true, automatically solve all oracle problems, prevent all collusion, or replace legal/compliance review. It does not guarantee project success. A proof package can be incomplete, misleading, or attacked. A validator set can be captured. A human authority can make a bad decision. An on-chain anchor can preserve a flawed receipt.

The contribution is more precise: GoalOS makes the work-verification process explicit, inspectable, challengeable, and settlement-gated. That is a major improvement over informal trust, but it is not omniscience.

15 Conclusion

Blockchain made settlement programmable. The next requirement is making work provable.

A transaction can be valid while the work behind it is weak. A DAO vote can pass while the evidence is incomplete. A grant can be paid while the milestone is ambiguous. An audit badge can exist while remediation is not replayable. An AI agent can produce persuasive output without earning authority. These are not edge cases; they are the everyday credibility problem of blockchain systems.

GoalOS Signoff Pro provides the missing institutional layer: mission, evidence, replay, validation, risk, authority, receipt, anchor, and Chronicle. Its public-safe demos show the right posture: no wallet, no value moved, no personal data, no overclaiming.

v28 and v29 should make the blockchain implication unmistakable: every serious blockchain project should be able to answer the public question, *where is the proof package?*

Operational standard

**Blockchain proves the transaction. GoalOS proves the work.
No Proof. No Trust. No Settlement.**

A Institutional adoption blueprint

A.1 The board-level decision

The board-level question is not whether every blockchain project can eliminate risk. It cannot. The decision is whether a project will continue to ask stakeholders for trust using fragmented evidence, or whether it will adopt a formal proof package that makes work reviewable before value moves.

Decision layer	Weak posture	GoalOS posture
Grant funding	Updates and screenshots	Evidence Docket and milestone receipt
Audit closure	"Fixed" statement	Finding-to-fix map, tests, replay, validator report
Protocol upgrade	Governance narrative	Scope, risk ledger, rollback plan, challenge window
Treasury payment	Vendor invoice	Mission Receipt bound to accepted evidence
AI-agent work	Agent transcript	ProofBundle, trace roots, validator quorum, human gate
RWA / external claim	Attestation headline	Provenance, evidence map, claim boundary, reviewer signoff

A.2 The credibility ladder

Table 7: GoalOS blockchain credibility ladder.

Level	Name	Behavior	Public trust
0	Narrative-only	Project publishes claims without structured evidence.	Low
1	Evidence-linked	Claims link to artifacts, but criteria and review are informal.	Basic
2	Docketed	Mission, claims, evidence, caveats, and public boundary are structured.	Moderate
3	Replayable	Independent reviewers can reproduce or inspect material results.	Strong
4	Receipted	Human authority signs a hash-bound decision record.	Institutional
5	Settlement-gated	Funds, reputation, governance, or capability move only after accepted receipt.	High

Level	Name	Behavior	Public trust
6	Chronicle-governed	Accepted proof becomes controlled memory; re-jected claims cannot propagate.	Compounding

Mandate language

Elite institutional standard. A credible project should not merely publish a contract address, audit logo, or roadmap. It should publish the proof object that explains why stakeholders should believe, fund, approve, or settle the work.

A.3 The public expectation

The expected public question becomes: *Where is the proof package?* This is a useful demand because it is simple, non-technical, and hard to fake at scale. It does not require every user to be an engineer. It requires the project to assemble its own evidence into a structure that engineers, auditors, governance delegates, and ordinary stakeholders can inspect.

B Reference proof package schema

Listing 2: Minimal public-safe proof package schema.

```

1 {
2   "mission_contract": {
3     "mission_id": "string",
4     "objective": "string",
5     "success_criteria": ["string"],
6     "failure_criteria": ["string"],
7     "risk_class": "low|medium|high|critical",
8     "allowed_actions": ["string"],
9     "denied_actions": ["string"],
10    "done_condition": "string"
11  },
12  "evidence_docket": {
13    "claims": [{"claim_id": "string", "status": "unsupported|mapped|validated|accepted"}],
14    "evidence_items": [{"evidence_id": "string", "hash": "sha256", "public_boundary": "public|redacted|private"}],
15    "claim_evidence_map": [{"claim_id": "string", "evidence_id": "string"}]
16  },
17  "proof_bundle": {
18    "trace_root": "sha256",
19    "output_root": "sha256",
20    "environment_root": "sha256",
21    "policy_root": "sha256",
22    "signature": "ed25519-or-equivalent"
23  },
24  "validation": {
25    "validator_quorum": "boolean",
26    "verdict": "accept|hold|challenge|reject",
27    "challenge_window": "open|closed|waived|failed"
28  },
29  "risk_ledger": {
30    "residual_risk": "low|medium|high|critical",
31    "rollback_ready": "boolean",
32    "known_caveats": ["string"]
33  },
34  "human_authority": {
35    "authority_role": "string",
36    "decision": "accept|hold|reject",
37    "timestamp": "ISO-8601"

```

```

38 },
39 "receipt": {
40   "receipt_id": "string",
41   "receipt_hash": "sha256",
42   "decision_state": "accepted|held|rejected|revoked",
43   "optional_anchor": "chain-and-transaction-or-null"
44 }
45 }

```

C Due-diligence scorecard

Table 8: Blockchain proof-package due-diligence scorecard.

Category	Score	Question
Mission clarity	0–5	Is the objective and done condition precise?
Evidence completeness	0–5	Are all claims mapped to evidence?
Replayability	0–5	Can the result be reproduced or independently inspected?
Validator independence	0–5	Are reviewers identified, qualified, and conflict-aware?
Risk disclosure	0–5	Are caveats, rollback, and residual risk documented?
Human authority	0–5	Is the accepting authority explicit?
Receipt integrity	0–5	Is the final decision signed and hash-bound?
Public boundary	0–5	Are privacy and public-data boundaries clear?
Challenge readiness	0–5	Is there a process for contesting the claim?
Settlement discipline	0–5	Does value move only after acceptance?

D Template mandate clauses

D.1 DAO grant clause

No milestone payment should be released until the grantee submits a public-safe proof package containing a mission contract, evidence docket, claims matrix, replay or inspection path, validator report, risk ledger, human or committee signoff, and signed mission receipt. The DAO may hold, challenge, reject, or request additional evidence when the proof package is incomplete.

D.2 Treasury payment clause

Treasury payments for external work should reference an accepted Mission Receipt. The receipt should bind to the approved evidence docket and validation report. Payment authorization should not rely on output-only claims, informal status updates, or unreviewed deliverables.

D.3 Protocol upgrade clause

Protocol upgrade proposals should include a GoalOS-style proof package: scope, rationale, implementation evidence, test and simulation results, audit findings, remediation map, rollback plan, risk ledger, challenge window, and human/governance acceptance record.

E Public copy bank

- **One-liner:** Blockchain proves the transaction. GoalOS proves the work.
- **Mandate:** Require the proof package before trust, payment, governance, or reputation moves.
- **Blockchain standard:** No Proof. No Trust. No Settlement.
- **DAO version:** No Evidence Docket, no milestone payout.
- **Agent version:** No ProofBundle, no agent settlement signal.
- **Upgrade version:** No replay, no release.
- **Investor version:** Show the receipt, not just the roadmap.

F Production-readiness checklist

1. Public demo pages are clearly labeled as demos or simulations.
2. No public demo page collects user data or connects wallets.
3. Every proof claim has a claim-boundary statement.
4. Every artifact has a hash and purpose.
5. Every acceptance decision references criteria.
6. Every high-risk action requires human authority.
7. Every settlement signal distinguishes synthetic readiness from live value movement.
8. Every on-chain reference is limited to hash/CID/timestamp unless production authorization is complete.
9. Every revocation or supersession path is documented.
10. Every public page answers: what does this prove, and what does it not prove?

References

- [1] MontrealAI. *The Signoff Institution: GoalOS Signoff Pro*. <https://montrealai.github.io/goalos-signoff-pro/>. Accessed 2026-07-01. 2026.
- [2] MontrealAI. *Mission 001 Reproducibility Packet: GoalOS Signoff Pro*. <https://montrealai.github.io/goalos-signoff-pro/mission-001.html>. Accessed 2026-07-01. 2026.
- [3] MontrealAI. *No user data by design: GoalOS Signoff Pro*. <https://montrealai.github.io/goalos-signoff-pro/no-user-data.html>. Accessed 2026-07-01. 2026.
- [4] MontrealAI. *GoalOS Proof-to-Settlement Control Lab*. <https://montrealai.github.io/goalos-signoff-pro/proof-settlement-lab.html>. Accessed 2026-07-01. 2026.
- [5] Ethereum.org. *Oracles*. <https://ethereum.org/developers/docs/oracles/>. Accessed 2026-07-01. 2026.
- [6] MontrealAI. *AGIALPHA is external: GoalOS Signoff Pro*. <https://montrealai.github.io/goalos-signoff-pro/agialpha.html>. Accessed 2026-07-01. 2026.
- [7] MontrealAI. *GoalOS AGIJobManager Ascension*. <https://montrealai.github.io/goalos-agijobmanager-ascension/>. Accessed 2026-07-01. 2026.
- [8] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. <https://bitcoin.org/bitcoin.pdf>. 2008.
- [9] Vitalik Buterin. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. <https://ethereum.org/whitepaper/>. 2014.
- [10] Lorenz Breidenbach et al. *Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks*. <https://research.chain.link/whitepaper-v2.pdf>. 2021.
- [11] W3C. *PROV-DM: The PROV Data Model*. <https://www.w3.org/TR/prov-dm/>. 2013.
- [12] OpenSSF. *SLSA: Supply-chain Levels for Software Artifacts*. <https://slsa.dev/>. 2026.
- [13] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. *in-toto: Providing farm-to-table guarantees for bits and bytes*. <https://in-toto.io/>. 2019.
- [14] Linux Foundation. *SPDX: Software Package Data Exchange*. <https://spdx.dev/>. 2026.
- [15] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. <https://www.nist.gov/itl/ai-risk-management-framework>. 2023.
- [16] National Institute of Standards and Technology. *Secure Software Development Framework (SSDF) Version 1.1: NIST SP 800-218*. <https://csrc.nist.gov/publications/detail/sp/800-218/final>. 2022.
- [17] MontrealAI. *goalos-signoff-pro GitHub repository*. <https://github.com/MontrealAI/goalos-signoff-pro>. Accessed 2026-07-01. 2026.
- [18] MontrealAI. *GoalOS Signoff Pro README*. <https://raw.githubusercontent.com/MontrealAI/goalos-signoff-pro/main/README.md>. Accessed 2026-07-01. 2026.